4.a Classes

Understanding Kotlin Class Fundamentals

Contents

1 **Class Declaration & Instances**
Declaring classes, class header, body, and creating instances

2 **Constructors**
Primary constructors, secondary constructors, and delegation

3 **Initializer Blocks**
init blocks, execution order, property initializers & validation

4 **Abstract Classes**
Abstract keyword, abstract members, subclass implementations

Class Declaration & Instances

Declaring a Class

- Use the `class` keyword followed by the class name
- Class header (type params, constructor) + body in { }
- Both header and body are optional

Creating Instances

- No `new` keyword — call the constructor like a function
- Classes are `final` by default — use `open` to allow inheritance

```
// Minimal class declaration
class Empty

val e1 = Empty()


// Class with properties
class Person(val name: String) {
    var age: Int = 0
}

// Creating instances
val person = Person("Alice")
println(person.name)  // Alice
println(person.age)   // 0
```

Primary Constructor

Syntax

- Part of the class header, after the class name
- The `constructor` keyword can be omitted if no annotations or visibility modifiers

Properties

- Use `val` (read-only) or `var` (mutable) to declare properties
- Can have default values
- Cannot contain any code — use `init { }` blocks instead

```
// constructor keyword optional
class Person2(
    val firstName: String,
    val lastName: String,
    var age: Int = 0
)

// Usage
val p = Person2("John", "Doe", 25)
```

```
// With annotations/modifiers
class Customer private constructor(
    name: String
) {
    companion object {
        fun create(name: String): Customer {
            return Customer(name)
        }
    }
}

val c = Customer.create("Jane")
```

Initializer Blocks

How init Blocks Work

- Prefixed with the `init` keyword inside the class body
- Can access primary constructor parameters
- Multiple *init* blocks allowed — run in order

Execution Order

- *Init* blocks and property initializers run in declaration order
- Execute before secondary constructor body
- Common use: data validation with `require()`

```
Output:
Init block: John Michael David Doe
First: John
Middle: Michael David
Last: Doe
```

```
class Person3(val name: String, val age: Int) {
    private val nameParts = name.split(" ")

    val first = nameParts.firstOrNull() ?: ""
    val middle = if (nameParts.size > 2)
        nameParts.subList(1, nameParts.size - 1)
                 .joinToString(" ")
        else
        ""

    val last = nameParts.lastOrNull() ?: ""

    init {
        require(age > 0)
    }

    init {
        println("Init block: $name")
        println("First: $first")
        println("Middle: $middle")
        println("Last: $last")
    }
}

val p3 = Person3("John Michael David Doe", 34)
```

Secondary Constructors

Syntax

- Declared inside class body with **`constructor`** keyword
- Useful for multiple initialization paths or Java interop

Delegation

- Must delegate to primary constructor via **`this`**
- Direct: calls primary constructor immediately
- Indirect: calls another secondary constructor
- All *init* blocks run before secondary constructor body

```
class Person4(val name: String) {
    var children: MutableList<Person4> = mutableListOf()

    // Secondary constructor
    constructor(
        name: String,
        parent: Person4
    ) : this(name) {
        parent.children.add(this)
    }
}

val parent = Person4("Alice")
val child = Person4("Bob", parent)

val aliceChildren = parent.children.map { it.name }

println("${parent.name}, children: $aliceChildren")
println("${child.name}, parent: ${parent.name}")
```

```
Output:
Alice, children: [Bob]
Bob, parent: Alice
```

Abstract Classes

What Are Abstract Classes?

- Cannot be instantiated directly
- Declared with the **`abstract`** keyword
- Implicitly **`open`** — no need for the open keyword

Key Features

- Can have both abstract and non-abstract members
- Abstract members have no implementation
- Subclasses must **`override`** all abstract members
- Can have constructors

```
abstract class Person5(
    val name: String,
    val age: Int
) {
    abstract fun introduce()
    fun greet() {
        println("Hello, $name")
    }
}

// Subclass
class Student(
    name: String,
    age: Int,
    val school: String
) : Person5(name, age) {
    override fun introduce() {
        println("I am $name, age $age, and I study at $school.")
    }
}

val student = Student("Charlie", 20, "University")
student.introduce()
student.greet()
```

```
Output:
I am Charlie, age 20, and I study at University.
Hello, Charlie
```

Practice Exercises

Solved

- **Exercise 1:** Create a `Book` class with title, author, and pages. Add an init block that validates pages > 0.
- **Exercise 2:** Create an abstract `Shape` class with an abstract `area()` function. Implement `Circle` and `Rectangle` subclasses.

Unsolved

- **Exercise 3:** Create a `BankAccount` class with primary and secondary constructors. Validate that initial balance is not negative.
- **Exercise 4:** Design an abstract `Vehicle` class with abstract `fuelType()` and a concrete `info()` method. Implement `Car` and `ElectricCar`.

Solution: Exercise 1

Exercise 1

- Create a **Book** class with title, author, and pages
- Add an **init** block that validates pages > 0

Key Concepts Used

- Primary constructor with **val** properties
- **init** block for validation logic
- **require()** for argument validation
- String templates for output

```kotlin
class Book(
    val title: String,
    val author: String,
    val pages: Int
) {
    init {
        require(pages > 0) {
            "Pages must be positive non zero"
        }
    }
}

// Usage
val book = Book("Kotlin in Action", "D. Jemerov", 360)
println("${book.title} by ${book.author} " +
        "with ${book.pages} pages")
```

```
Output:
Kotlin in Action by D. Jemerov with 360 pages
```

Solution: Exercise 2

Exercise 2

- Create an abstract **Shape** class with abstract **area()**
- Implement **Circle** and **Rectangle** subclasses

Key Concepts Used

- **abstract** class and function declarations
- Inheritance with **:** syntax (no **open** needed)
- **override** keyword for implementation
- Using **Math.PI** and polymorphism

```
abstract class Shape {
    abstract fun area(): Double
}

class Circle(val radius: Double) : Shape() {
    override fun area(): Double =
        Math.PI * radius * radius
}

class Rectangle(val w: Double, val h: Double) : Shape()
{
    override fun area(): Double = w * h
}

// Usage
val shapes = listOf(Circle(5.0), Rectangle(4.0, 6.0))
shapes.forEach {
    println("Area: ${it.area()}")
}
```

```
Output:
Area: 78.53981633974483
Area: 24.0
```